

QualiJournal 관리자 주요 기능 문제 분석 및 해결책

1. 작업 취소 기능(cancelJob) 문제

문제 원인: 취소 버튼을 눌러도 백그라운드 작업이 계속 진행되고 UI에도 즉각 반영되지 않습니다. 프론트엔드 `cancelJob()` 함수는 `/api/tasks/{job_id}/cancel` 엔드포인트를 호출하지만, 호출 후 별도의 UI 업데이트를 하지 않아 모달이 닫히지 않고 작업이 지속되는 것처럼 보입니다 ①. 서버 측에서는 `cancel_task` 함수가 해당 작업 객체의 `_cancel` 플래그만 True로 설정하고 끝나며 ②, 실제 태스크 스레드에서는 이 플래그를 확인해 프로세스를 `kill()` 한 뒤 `! canceled` 로그만 남기고 종료(return 1)합니다 ③ ④. 그러나 이 경우 태스크의 최종 `status`를 “canceled”로 표시하지 않고 `exit_code`가 0이 아니므로 “error”로 처리하기 때문에 SSE 스트림의 종료 이벤트에서도 상태가 `canceled`로 나타나지 않습니다. 결과적으로 작업이 취소돼도 UI에는 `<END>` `error`로 표시되어 사용자가 취소되었음을 인지하기 어렵고, 모달도 자동으로 닫히지 않아 **취소가 제대로 적용되지 않은 것처럼 보이는 현상**이 발생합니다.

해결 방안:

- **백엔드 수정:** `Task._cancel` 플래그가 True인 경우 태스크 종료 시점에 `task.status`를 명시적으로 “canceled”로 설정해야 합니다. 예를 들어 `_run_task` 함수의 `finally` 블록에서, `_cancel`이 True이면 `task.status = "canceled"`로 지정합니다. 이를 통해 SSE 스트림의 종료 이벤트(`event: end`)에서 `data: canceled`가 전달되어 UI에서도 취소 상태를 명확히 인식할 수 있습니다.
- **프론트엔드 수정:** 취소 버튼 동작 후 UI 갱신을 추가합니다. `cancelJob()` 함수에서 취소 API 호출 후 즉시 SSE 스트림을 닫고(`stopStream()` 호출) 현재 작업 상태를 정리하도록 합니다. 또한 필요하면 모달을 닫거나(`closeModal()`), 취소 요청이 제출되었다는 메시지를 로그/토스트에 남겨 사용자에게 피드백을 줍니다. 이렇게 하면 취소 버튼 클릭 시 실시간으로 UI에 취소가 반영됩니다.

확인 방법: 긴 작업(예: 키워드 특집 수집)을 시작한 뒤 취소 버튼을 눌러 봅니다. 수정 전에는 로그에 `<END> error`로 끝나고 모달이 닫히지 않았지만, 수정 후에는 `<END> canceled` 메시지가 출력되고 모달이 닫혀야 합니다. 또한 취소 후 새 작업을 바로 시작할 수 있는지 확인합니다 (버튼 비활성화 해제 등).

질의 템플릿: “작업 취소 버튼을 눌렀는데도 백그라운드 작업이 멈추지 않습니다. 취소 기능의 문제점과 해결책을 무엇 인가요?”

2. 키워드 특집 ‘수집만’ 버튼(btn-collect-only) 문제

문제 원인: 키워드 특집 기능에서 “수집만” 기능이 UI에 표시되었으나, 해당 버튼을 눌렀을 때 아무 반응이 없습니다. 이는 **프론트엔드에 해당 버튼의 이벤트 핸들러가 구현되지 않았거나**, 백엔드에 이를 처리할 API 경로가 없기 때문입니다. 현재 코드에서는 키워드 특집 실행 시 전체 프로세스(수집→최적기사 선정→발행)를 진행하는 버튼만 있고 ⑤, “수집만”에 해당하는 별도 함수나 엔드포인트가 없습니다. 즉, **수집 단계만 수행하는 로직이 빠져있는 것**이 문제입니다.

해결 방안:

- **백엔드 구현:** 키워드 기사 수집만 수행하는 엔드포인트를 추가합니다. 예를 들어 `POST /api/flow/keyword-collect` 또는 `/api/collect-keyword` 등의 경로를 만들어, 내부에서 `orchestrator.py --collect-keyword <키워드>` 명령만 실행하도록 합니다. 현재 `/api/flow/keyword` 엔드포인트는 수집 후 곧바로 승인·발행 단계까지 수행하고 있는데 ⑥, “수집만” 요청 시에는 `--collect-keyword`만 호출하고 나머지는 생략하거나, 또는 `use_external_rss` 등 추가 매개변수로 동작을 달리하도록 변경할 수 있습니다.
- **프론트엔드 구현:** ‘수집만’ 버튼에 대한 클릭 핸들러를 구현해 위 신규 엔드포인트를 호출하도록 합니다. 예를 들어

`onclick="runKeywordCollectOnly()"` 함수를 만들고, 이 함수에서 `fetch('/api/flow/keyword-collect', {...})`를 호출하여 수집 작업만 수행하도록 합니다. 실행 결과는 기존처럼 작업 진행 모달이나 SSE로 그 패널로 피드백하면 됩니다.

확인 방법: “수집만” 버튼 클릭 시 실제로 키워드에 대한 기사 수집만 이루어지고, 이후 자동 발행은 되지 않는지 확인합니다. 백엔드 로그에서 orchestrator가 `--collect-keyword`까지만 수행하고 `--approve-keyword`나 `--publish-keyword`를 실행하지 않는지 모니터링합니다. 또한 수집 완료 후 **Ready 목록** 등에 수집된 기사들이 나타나는지 확인합니다.

질의 템플릿: “키워드 특집에서 ‘수집만’ 기능이 동작하지 않습니다. 이 버튼을 눌렀을 때 반응이 없는 원인과 구현 방법을 알려주세요.”

3. 데이터 새로고침 버튼(btnRefresh, btnKpiRefresh) 문제

문제 원인: 사이드바에 있는 “새로고침” 및 “KPI 새로고침” 버튼을 눌러도 아무 일도 일어나지 않습니다. 이는 해당 버튼들에 이벤트 핸들러가 연결되어 있지 않기 때문입니다. 실제 코드에서 이 버튼들은 `data-action` 속성만 정의되어 있고(`status:refresh`, `status:kpi`) 별도 `onclick`이나 이벤트 리스너가 없습니다⁷. 따라서 버튼 클릭이 `refresh()`나 `loadReady()` 함수를 호출하지 않아 **UI 갱신이 이루어지지 않는 것**입니다.

해결 방안:

- **프론트엔드 수정:** 버튼에 대응하는 이벤트 바인딩을 추가합니다. 간단히는 HTML에 `onclick` 속성을 추가하는 방법이 있고, 또는 자바스크립트에서 `DOMContentLoaded` 후

`document.getElementById('btnRefresh').onclick = loadReady;` 식으로 연결할 수 있습니다. - “**새로고침**” 버튼(btnRefresh): 이 버튼은 **Ready 목록을 다시 불러오는 기능**으로 구현되어야 합니다. 따라서 클릭 시 `loadReady()` 함수를 호출하도록 합니다. `loadReady()` 함수는 `/api/items?state=ready` 엔드포인트를 GET으로 불러와 Ready 상태 기사 목록을 갱신하는 로직이므로, 이 호출로 최신 Ready 목록을 화면에 렌더링합니다^{8 9}.

- “**KPI 새로고침**” 버튼(btnKpiRefresh): 이 버튼은 **통계 지표(KPI)를 갱신**하는 역할입니다. 클릭 시 `refresh()` 함수를 호출하도록 합니다. `refresh()` 함수는 `/api/status` API를 호출해 전체 선택/승인/발행 수치와 게이트 통과 여부 등을 받아와 화면의 KPI 표시를 업데이트하는 로직입니다^{10 11}. 이 함수를 버튼에 연결하여 KPI 수치를 수동 새로고침할 수 있게 합니다. 필요하면 `runWithUiFeedback`으로 래핑하여 사용자에게 완료 토스트를 보여주는 것도 가능합니다.

확인 방법: 각 버튼 클릭 시 콘솔 에러가 발생하지 않고, **화면의 숫자나 목록이 갱신**되는지 확인합니다. 예를 들어 “새로고침” 버튼을 누르면 좌측 Ready 목록이 최신 상태로 재로드되어야 하고(신규 Ready 기사 포함 여부 등), “KPI 새로고침” 버튼을 누르면 상단 KPI 표시(선정된 기사 수, 승인/발행 수, 커뮤니티/키워드 기사 수 등)가 업데이트되어야 합니다.

질의 템플릿: “관리자 화면에서 ‘새로고침’ 버튼과 ‘KPI 새로고침’ 버튼을 눌러도 반응이 없습니다. 어떤 문제가 있는지, 어떻게 수정하면 좋을까요?”

4. Ready 목록 불러오기 기능(loadReady) 문제

문제 원인: Ready 상태 기사 목록을 불러오는 기능이 동작하지 않습니다. 프론트엔드에서는 `loadReady()` 함수가 Ready 목록을 표시하도록 되어 있지만⁸, 서버 측에 해당 요청을 처리하는 `/api/items?state=ready` 경로가 없으면 당연히 실패하게 됩니다. 질문에서도 서버에 `/api/items?state=ready` 미구현을 지적하고 있으며, 이는 이전 버전 서버에 해당 엔드포인트가 없었거나 동작이 누락되었음을 의미합니다.

해결 방안:

- **백엔드 구현:** `/api/items` GET 엔드포인트를 구현하여 `state` 파라미터에 따라 기사 목록을 반환해야 합니다. 예시 구현으로, `state=ready` 일 경우 현재 발행 준비 완료 상태의 기사들을 반환하도록 합니다. 이미 최신 코드에서는 `api_items` 함수가 존재하며, `state` 값에 따라 분기 처리하고 있습니다: `state=="ready"` 인 경우 `selected_keyword_articles.json` 또는 `selected_articles.json` 등을 읽어 **Ready 상태 기사 목록**을 모아 반환합니다¹². 이처럼 백엔드에 Ready 목록 조회 로직을 추가/수정하여, 프론트엔드가 요청할 때 올바른 데이터가 나오도록 합니다.

- 구현 시, 만약 별도로 Ready 상태를 명시하는 필드가 없다면, 수동으로 선정(approved)되었지만 아직 발행하지 않은 기사들을 Ready로 간주할 수 있습니다. (현재 코드에서는 기사 객체의 `state` 필드가 `"ready"` 인 경우와, 또는 `selected`/`approved` 플래그가 True인 경우를 필터링하고 있습니다¹³.) 이 규칙에 따라 적절히 JSON 필터링을 하면 됩니다. - **프론트엔드 확인:** 위 3번의 수정대로 `btnRefresh`가 `loadReady()`를 호출하도록 연결하면, 사용자가 Ready 목록 새로고침을 눌렀을 때 백엔드의 `/api/items?state=ready`가 호출됩니다. 이제 백엔드가 구현되었으므로 정상적인 응답(JSON)이 와서 `renderReadyList()`를 통해 목록이 갱신될 것입니다¹⁴¹⁵.

확인 방법: Ready 목록에 새로운 기사를 추가하거나 상태 변화를 준 뒤, 새로고침 기능으로 해당 목록이 업데이트되는지 확인합니다. 예를 들어 새로운 키워드 특집 작업을 수행하여 Ready 상태 기사가 생성된 후, 새로고침을 눌렀을 때 그 기사가 목록에 나타나야 합니다. 혹은 Ready 기사 하나를 발행한 뒤 새로고침하면 해당 기사가 목록에서 빠지는지도 확인합니다. (이 과정에서 `/api/items?state=ready` 호출에 대한 서버 응답이 정상이고 오류가 없는지 확인하세요.)

질의 템플릿: “Ready 기사 목록이 UI에 표시되지 않습니다. 서버와 프론트엔드에서 Ready 목록 불러오기 기능을 어떻게 구현하거나 수정해야 할까요?”

5. 보고서·요약 기능 문제 (빈 MD 파일 생성 등)

문제 원인: 뉴스 요약 및 보고서 생성 기능이 정상 동작하지 않아, `/api/enrich/keyword`나 `/api/enrich/selection`을 호출해도 내용이 채워진 Markdown 파일이 나오지 않습니다. 현재 백엔드 구현을 보면, 해당 엔드포인트를 호출하면 `tools/enrich_cards.py` 같은 요약 스크립트를 실행해야 함에도 불구하고 실제로는 그러한 처리가 빠져 있습니다. 코드상 `/api/enrich/keyword` 실행 흐름을 보면: `selected_keyword_articles.json` 등의 데이터를 읽고 제목만 나열한 MD 파일을 바로 생성해버립니다¹⁶. 즉, **AI 요약 모델을 호출하는 로직이 수행되지 않아** 각 기사에 `summary` 필드가 비어 있고, 그 결과 MD 파일에도 제목 외에 요약문이 없는 빈 내용이 생성됩니다. 이 기능에 **입력 검증 부족**도 지적되는데, 예를 들어 사용자로부터 키워드 입력이나 날짜를 받아야 하는 경우 그 체크가 미흡했던 것으로 보입니다 (다행히 최신 프론트 코드에서는 키워드 미입력 시 토스트 오류를 내도록 일부 처리되어 있습니다¹⁷).

해결 방안:

- **백엔드 요약 로직 추가:** 요약/번역을 수행하는 **AI 모델 호출 부분을 구현**해야 합니다. 설계 문서에 따르면 `/api/enrich/*` 호출 시 내부적으로 `tools/enrich_cards.py` 스크립트를 실행하도록 되어 있습니다. 따라서 백엔드 `enrich_keyword` 함수에서 Markdown을 만들기 전에 `enrich_cards.py`를 실행해야 합니다. 구체적으로, 1. `subprocess`나 `_run_py()` 유틸함수를 통해 `tools/enrich_cards.py`를 호출하여 요약 작업을 수행합니다. 이 스크립트는 `selected_keyword_articles.json`의 각 기사에 대한 영어 요약(`summary`)과 한글 번역(`summary_ko` 등)을 생성해 JSON 파일을 업데이트한다고 가정합니다. 2. 스크립트 실행 후, JSON 파일을 다시 읽어 요약 필드들이 채워졌는지 확인합니다. 그런 다음 `_generate_summary_md`를 호출하면 각 기사에 `summary`나 `ko_summary` 내용이 있기 때문에 **Markdown 파일에 요약문이 포함**됩니다¹⁸. 3. 요약이 완료된 JSON을 발행용(`selected_articles.json`)과 동기화하거나, 필요시 번역본 처리 등 부가 작업도 고려합니다.

• **입력값 검증:** 프론트엔드에서 보고서/요약을 요청할 때 필요한 값들이 채워졌는지 확인합니다. 예를 들어 **키워드 요약**의 경우 키워드 입력이 비어 있다면 요청을 보내지 않도록 처리하는데(현재 `enrichKeyword()`에서

구현됨 17), 혹시 누락된 부분이 있다면 추가합니다. **일일 보고서 생성(Report)**도 날짜나 키워드가 없으면 기본값을 채우는 등의 처리를 하여 잘못된 입력으로 API를 호출하지 않도록 합니다.

확인 방법: 실제 외부 API 키(API_KEY)를 설정한 상태에서 **요약/번역 기능을 실행**해 봅니다. `/api/enrich/keyword` 호출 후 `archive/enriched/` 디렉토리에 생성된 Markdown 파일을 열어보면, 각 기사 제목 아래에 - 요약: 문구와 생성된 요약문이 포함되어야 합니다 19. 또한 `/api/enrich/selection`을 호출할 경우 선택된(승인된) 기사들만 대상으로 요약 파일이 만들어지는지 확인합니다. 입력 검증 측면에서는, 키워드 입력 없이 요약 버튼을 눌렀을 때 토스트로 “키워드를 입력하세요”가 뜨는지 등을 테스트합니다.

질의 템플릿: “키워드 뉴스 요약/번역 기능을 실행하면 Markdown에 내용이 없습니다. 요약 기능이 빈 결과를 내는 이유와, 이를 정상 동작시키기 위한 구현 방안을 설명해주세요.”

6. 게이트 조절 기능 문제 (슬라이더 UI 즉각 반영 안됨)

문제 원인: 발행 게이트 임계값(예: 15건)을 조절하는 슬라이더를 움직였을 때, **UI 표시가 바로 변하지 않거나 부자연스럽습니다.** 현재 구현을 보면 슬라이더(`id="gateSlider"`)의 `oninput` 이벤트로 **현재 값 숫자만** 업데이트하고(`gateVal` 스펠에 반영) 20, “통과/미통과” 상태(`gatePass`)는 즉시 변경하지 않습니다. `gatePass`는 게이트 임계값과 현재 승인된 기사 수를 비교해 결정하는데, 이는 슬라이더 변경 후 `/api/config/gate_required` PATCH 요청이 완료되고 다시 `/api/status`를 새로 고침해야만 갱신됩니다 21 11. 따라서 슬라이더를 움직이는 순간에는 게이트 통과 여부 표시가 이전 상태로 남아 있어 **사용자에게 즉각적인 피드백이 부족**합니다.

해결 방안:

- **프론트엔드 즉시 반영:** 슬라이더 `oninput` 이벤트 핸들러를 확장하여, 값 숫자뿐 아니라 **게이트 통과 여부 표시도 실시간으로 업데이트**합니다. 현재 승인된 기사 수(예: `selection_approved`)는 직전에 `/api/status`를 불러왔을 때 알고 있으므로, 이를 전역 변수나 DOM에서 가져와 비교하면 됩니다. 예를 들어:

```
oninput="document.getElementById('gateVal').textContent=this.value;
var approved = Number(document.getElementById('selApproved').textContent||0);
document.getElementById('gatePass').textContent = (approved >= this.value ? '통과':'미통과');"
```

이렇게 하면 슬라이더를 드래그하는 동안에도 **게이트 통과:** 레이블이 즉시 변경되어, 사용자가 몇으로 설정해야 통과 되는지 직관적으로 볼 수 있습니다.

- **PATCH 후 처리:** 이미 구현된 `updateGate()` 함수는 PATCH 요청 성공 후 `refresh()`를 호출해 게이트 정보를 새로 고칩니다 21. 이 부분은 그대로 두되, 혹시 **슬라이더를 빠르게 연속 조작**할 경우 불필요한 호출이 많아질 수 있으므로, 필요하면 입력 지연(throttle) 처리를 고려할 수 있습니다. (일반적으로는 현재 방식으로도 무방합니다.)

확인 방법: 슬라이더를 좌우로 움직일 때마다 **오른쪽의 “게이트 통과/미통과” 표시가 바로 바뀌는지** 확인합니다. 예를 들어 현재 승인된 기사 수가 10개이고 게이트 기본값이 15라면 처음 “미통과”였다가, 슬라이더를 10 이하로 내리면 바로 “통과”로 바뀌어야 합니다. 슬라이더를 놓으면 PATCH 요청이 보내지고, 토스트 “게이트 저장: X”가 뜬 후 서버에서 최종 확정된 값으로 한 번 더 업데이트되는지 확인합니다.

질의 템플릿: “관리자 화면에서 게이트 임계값을 조절해도 ‘통과/미통과’ 표시가 바로 안 바뀝니다. 슬라이더 UI와 게이트 상태 표시를 즉시 연동시키려면 어떻게 해야 하나요?”

7. 외부 RSS 사용 옵션 누락 문제 (키워드 수집 경로)

문제 원인: 키워드 특집 기사 수집 시 **외부 RSS 소스 포함 여부를 선택할 수 없는 문제**입니다. 현재 키워드 수집 백엔드 로직을 보면, **동기식 경로**(`/api/flow/keyword`)에서는 `use_external_rss` 플래그를 지원하여 True이면

외부 RSS까지 포함하도록 orchestrator를 호출하지만 ²², **비동기 태스크 경로**(`/api/tasks/flow` 통해 실행)는 이 옵션을 전혀 전달하지 않습니다. 즉, SSE 기반으로 `startKeyword()`를 실행하면 기본적으로 외부 RSS를 제외한 채 수집이 이루어집니다 ²³. 프론트엔드에도 외부 RSS 토글 UI는 없고, 설정 기본으로 사용하려 해도 태스크 실행 경로에 반영되지 않으므로 **외부 RSS가 누락된 수집**이 이뤄집니다.

해결 방안:

- **백엔드 수정:** 태스크 매니저 경로에서도 외부 RSS를 사용할 수 있도록 수정합니다. 간단한 해결로, **키워드 작업 실행 시 항상 외부 RSS를 포함하도록** 할 수 있습니다. `Task.kind == "keyword"` 부분의 명령 실행에 `--use-external-rss` 옵션을 추가하면, SSE 방식이든 동기 방식이든 동일하게 외부 RSS를 수집합니다. 예를 들어 `_run_task` 내부를 아래처럼 바꿉니다:

```
elif task.kind == "keyword":
    kw = task.args[0] if task.args else ""
    ...
    rc1 = run_cmd([py, str(ORCH), "--collect-keyword", kw, "--use-external-rss"])
    ...
```

이렇게 하면 `/api/tasks/flow`로 시작된 키워드 작업도 외부 RSS를 포함하게 됩니다.

만약 **옵션으로 토글**할 수 있게 하려면, 프론트엔드에서 “외부 RSS 사용” 체크박스를 제공하고, `/api/tasks/flow` 요청에 해당 플래그를 넣어야 합니다. 이를 위해 `FlowReq` 모델과 `create_flow` 처리에 `use_external_rss`를 반영하는 개선이 필요합니다. 하지만 현재 요구사항에서는 기본적으로 포함시키는 편이 간단한 해결입니다.

- **프론트엔드 확인:** 앞단에서 키워드 특집 시작(`runKeyword`)은 이미 기본적으로 외부 RSS를 True로 하여 `/api/tasks/flow`를 호출하고 있었습니다 ²⁴. 그러나 이 코드는 **동기 모드 풀백**일 때만 쓰이고, 실제 SSE 모드에서는 별도 분기가 없습니다. 백엔드가 위처럼 수정되면 프론트엔드 변경 없이도 동작이 일치하게 되므로, 추가 UI 요소가 꼭 필요하지는 않습니다 (필요하다면 옵션 UI를 추가 구현).

확인 방법: 키워드 특집 실행 후 **수집된 기사 출처를 확인**합니다. 외부 RSS가 포함되었다면, 예컨대 bing이나 구글 뉴스 등 외부 RSS에서 가져온 기사들도 `selected_keyword_articles.json`에 존재해야 합니다. 또한 orchestrator 실행 로그에 `--use-external-rss` 옵션이 포함되어 출력되는지 확인합니다. 이전에는 키워드 특집 수집 시 내부 데이터베이스 또는 한정된 소스만 사용했다면, 수정 후에는 외부 뉴스도 함께 수집되는 것을 테스트해 볼 수 있습니다.

질의 템플릿: “키워드 뉴스 수집 시 외부 RSS를 포함하지 못하고 있습니다. 외부 RSS까지 함께 수집하려면 코드 수정을 어떻게 해야 하나요?”

8. 승인 UI 링크(startApprove) 문제

문제 원인: **승인 UI 열기** 버튼을 눌렀을 때, 백엔드가 빈 문자열 링크를 반환하면 아무 동작도 하지 않는 문제가 있습니다. 코드 상으로 `/api/approve-ui/start` 호출 후 `d.ui_url` 값을 받아 창을 열도록 되어 있는데, `ui_url`이 `""` (빈 문자열)인 경우 자바스크립트에서 falsy로 취급되어 `API_BASE`로 대체됩니다 ²⁵. 그러면 현재 관리도메인 (예: admin.standardai.co.kr)으로 새 탭을 열게 되어 **의도한 승인 UI가 뜨지 않고 아무 변화가 없는 것처럼 보이게** 됩니다.

해결 방안:

- **백엔드 측:** `/api/approve-ui/start` 엔드포인트가 빈 문자열을 주지 않도록 개선하는 것이 근본 해결입니다. 예를 들어 승인 UI URL을 환경변수 등으로 설정하고, 없다면 아예 `ui_url` 키를 응답해서 빼버리거나 null을 주는 식으로 처리합니다.

- **프론트엔드 측:** 빈 링크에 대한 처리를 보완합니다. `startApprove()` 함수 내에서 `d.ui_url`이 없거나 빈 문

자열이면, `window.open()` 을 호출하지 말고 사용자에게 “승인 UI 링크가 설정되지 않았습니다” 등의 안내를 합니다. 구현상으로는, `var ui = d.ui_url || API_BASE;` 이 부분을 수정하여 `d.ui_url` 이 `""` 인 경우를 걸러냅니다. 예를 들어:

```
var ui = (d.ui_url && d.ui_url.trim() !== "") ? d.ui_url : null;
if(ui){
    window.open(ui, '_blank', 'noopener');
} else {
    showToast('승인 UI 링크를 확인할 수 없습니다', 'err');
}
```

이렇게 하면 빈 링크일 때는 새 탭을 열지 않고 사용자에게 실패를 알릴 수 있습니다.

확인 방법: `/api/approve-ui/start` 가 의도한 UI 링크를 반환하지 않는 상황(예: 개발 환경에서 미설정)에서도 버튼 클릭 시 오류 안내가 표시되는지 확인합니다. 그리고 실제로 링크가 올바르게 설정된 환경에서는 새 탭으로 해당 UI가 열리는지 테스트합니다. (예: `d.ui_url` 로 받은 URL이 제대로 `window.open` 으로 열리는지 확인). 만약 API가 항상 `ui_url` 을 주도록 수정했다면, 빈 문자열 상황을 시뮬레이션하기 위해 임시로 서버 응답을 빈 값으로만 들어보는 것도 방법입니다.

질의 템플릿: “관리자에서 ‘승인 UI 열기’ 버튼을 눌러도 아무 반응이 없습니다. 빈 링크일 때 처리를 포함해 이 기능을 어떻게 개선할 수 있을까요?”

9. Cloud Run 배포 환경 문제 (오케스트레이터 누락, .gcloudignore 설정 오류, 트래픽 미할당)

문제 원인: 배포한 Cloud Run 환경에서 여러 가지 설정 미흡으로 서비스가 정상 동작하지 않았습니다. - **orchestrator.py 누락:** 컨테이너 이미지에 핵심 스크립트인 `orchestrator.py` 가 포함되지 않았습니다. 이는 Cloud Build 시 빌드 컨텍스트나 ignore 설정 문제로 해당 파일이 업로드되지 않았기 때문입니다. 실제 서버 로그를 보면 `orchestrator` 실행 시 “`orchestrator.py not found in image`”라는 오류를 출력하고 있습니다²⁶. 이는 `.dockerignore` 또는 `.gcloudignore`에서 파일이 제외되었거나, 배포 경로를 잘못 지정한 경우 발생합니다. - **.gcloudignore 설정 오류:** `.gcloudignore` 파일이 없다면 기본적으로 `.gitignore`를 따르는데, 이로 인해 정적 파일이나 필요 파일이 빌드에서 제외되었을 수 있습니다²⁷²⁸. 예컨대 `.gitignore`에 `*.py`나 `tools/` 폴더가 무시되어 있었다면, `orchestrator.py`나 `index.html` 같은 파일이 빌드에 포함되지 않았을 가능성이 있습니다. - **Cloud Run 트래픽 미할당:** 새 Revision으로 배포는 되었지만 트래픽이 할당되지 않아 실제 사용자 요청이 구 버전에 머무르는 문제가 있었습니다. Cloud Run은 기본적으로 새 배포에 100% 트래픽을 주지만, 만약 `--no-traffic` 옵션을 사용했거나 수동 승격을 해야 하는 설정이었다면 최신 버전이 서비스되지 않습니다.

해결 방안:

- **오케스트레이터 파일 포함:** 우선 배포 컨텍스트에 `orchestrator.py`가 포함되도록 수정합니다. Dockerfile의 COPY 경로가 올바른지 확인하고, `.dockerignore`에서 `orchestrator.py`나 그 폴더를 제외하고 있지 않은지 점검합니다²⁹. 소스 배포(`gcloud run deploy --source .`)를 사용한다면, `.gcloudignore` 규칙을 확인하여 `.gitignore`에 의해 `orchestrator.py`가 무시되지 않도록 합니다²⁸. 예를 들어 `.gitignore`에 `admin/orchestrator.py`가 들어있다면 `.gcloudignore` 파일을 만들어 `!admin/orchestrator.py`를 추가하여 포함시킵니다.

- **정적 파일 누락 확인:** 마찬가지로 `index.html`, JS, CSS 등 정적 리소스가 이미지에 포함되도록 `.gcloudignore`/`.dockerignore`를 조정합니다²⁷³⁰. 특히 `.dockerignore`에 일반적으로 `**/__pycache__`나 `.git` 등을 제외하되, `index.html`이나 `tools/` 폴더를 제외하면 안 됩니다. 필요하다면

`.dockerignore` 에서 해당 라인을 제거하거나 주석 처리합니다.

- **Cloud Run 트래픽 할당:** 배포 직후 **최신 리버전이 트래픽을 받고 있는지** 확인해야 합니다. GCP 콘솔의 Cloud Run 서비스 상세에서 Revision 목록을 보거나, CLI로 `gcloud run services describe <서비스명>` 명령을 실행하면 현재 트래픽 분배 상황을 볼 수 있습니다. 최신 리버전에 `0%` 로 되어 있다면 새 버전이 노출되지 않습니다. 해결책은, 배포 시 `--traffic=latest=100` 옵션을 사용하거나 ³¹, 배포 후 아래와 같이 명령어로 트래픽을 이동합니다:

```
gcloud run services update-traffic <서비스명> --to-revisions <최신리버전>=100
```

이를 실행하면 최신 리버전으로 100% 트래픽이 할당되어 사용자에게 새로운 코드가 적용됩니다.

- **기타:** 첫 배포 시 Cloud Build/Run API 활성화, PORT 환경변수 설정 등도 함께 챙겨야 합니다 ³². 특히 PORT는 이미 코드에서 `uvicorn` 실행 시 사용하고 있으므로 문제가 없지만, `.env` 파일에 `ADMIN_TOKEN` 등의 값이 설정됐는지도 점검합니다.

확인 방법: 수정 후 **Cloud Run에 다시 배포**하고, 배포 로그에 `orchestrator.py`와 정적 파일들이 포함되었는지 확인합니다. (Dockerfile 끝에 `RUN ls -R /app`를 넣어보는 방법으로 이미지 내용을 확인할 수 있습니다 ³³.) 배포 완료 후 Cloud Run 콘솔에서 **Revision에 트래픽이 할당**되었는지 확인하세요 - 최신 리버전에 ✓ 표시가 있고 이전 것이 X로 표시되면 성공입니다. 그런 다음 실제 웹 UI에 접속해 변경사항(예: 위에서 수정한 기능들)이 반영되었는지 테스트합니다. 필요하다면 브라우저 캐시를 무시하고 새로고침하여 최신 정적 파일을 받아오도록 합니다 ³⁴.

질의 템플릿: “Cloud Run 배포 후에도 코드 수정 사항이 반영되지 않습니다. `orchestrator.py` 누락, `.gcloudignore` 설정, 트래픽 할당 측면에서 무엇을 점검하고 해결해야 할까요?”

각 문제를 위와 같이 하나씩 점검하고 수정함으로써, QualiJournal 관리자 사이트의 핵심 기능들이 의도대로 동작하도록 개선할 수 있습니다. 필요한 수정 포인트를 반영한 후에는 충분한 테스트를 거쳐 배포하며, 배포 과정에서 **필수 파일 포함과 트래픽 할당**을 재확인해야 합니다. 이렇게 하면 나열된 9가지 문제를 모두 해결할 수 있습니다. ⁴ ²⁶

¹ ⁵ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹⁴ ¹⁵ ¹⁷ ²⁰ ²¹ ²⁴ ²⁵ `index.html`

<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/index.html>

² ³ ⁴ ⁶ ¹² ¹³ ¹⁶ ¹⁸ ¹⁹ ²² ²³ ²⁶ `server_quali.py`

https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/server_quali.py

²⁷ ³² 1015_1QualiJournal 관리자 시스템 Cloud Run 배포 및 도메인 연결 가이드.pdf

<file:///file-8y9Mz838Nti1LV8xWsXCq5>

²⁸ ²⁹ ³⁰ ³¹ ³³ ³⁴ 1022_1Cloud Run 배포 확인 및 최신 리버전 트래픽 적용 가이드.pdf

<file:///file-7X7JDAg5PPzVgjeeR8DG8V>